

RPA als Steuerungs- & Risikohebel

Plattformüberblick, Attended vs. Unattended –
verantwortete Automation unter Unsicherheit.

Lehrskript – Tag 11 von 16

Lernpfad:
Wissen → Anwendung → Management

Dozent:
Can Siebert

Bearbeitungsdauer:
1 Kurstag

Format:
Theorie · Fallstudie · Coaching

Lernziele dieses Tages

Robotic Process Automation (RPA) ist heute eines der häufigsten Werkzeuge zur Effizienzsteigerung in Verwaltungs- und Servicebereichen – und gleichzeitig eine der häufigsten Quellen für Schatten-IT, Compliance-Vorfälle und unkontrollierte Skalierung. Heute lernen Sie, RPA als *Steuerungs- und Risikohebel* zu denken: einzuordnen, was sich für Automation eignet, die richtige Form (attended oder unattended) bewusst zu wählen und Entscheidungen unter Zielkonflikten verantwortungsfähig zu treffen.

L1 – Wissen (Reproduktion und Einordnung)

- RPA-Eignungskriterien benennen: regelbasiert, stabiler Prozess, klarer Input/Output, hohes Volumen, geringe Ausnahmen.
- Plattformüberblick einordnen: Studio/Designer, Orchestrierung, Queues, Credentials, Monitoring – toolneutral.
- Unterschied attended vs. unattended verstehen: Betriebsmodell, Skalierungs- und Governance-Konsequenzen.
- Nutzen und Risiken von RPA aus Senior-Brille einordnen: Schatten-IT, Credential-Risiko, fragile UI-Automation, Audit Trail.

L2 – Anwendung (Transfer auf konkrete Situationen)

- Aus einer Use-Case-Liste die Eignung mit Scoring 1–5 bewerten und priorisieren.
- Eine Stufenstrategie attended → unattended entwerfen, inkl. Readiness-KPIs.
- Einen Bot-Flow mit Validierung, Verarbeitung, Logging und Exception Queue konzipieren.
- Audit-Trail-Minimum und KPI-Set (Produktivität, Qualität, Risiko) auf einen Fall anwenden.

L3 – Management (Entscheidung und Verantwortung)

- Ein RPA-Portfolio mit Kriterien Value/Risk/Effort und Kill Criteria steuern.
- Governance leicht, aber verbindlich verankern (Bot Owner, Process Owner, Security, Operations).
- Security- und Compliance-Guardrails (Least Privilege, SoD, Vault) als Pflichtprogramm setzen.
- Entscheidungen unter Unsicherheit (welche Use Cases zuerst, Unattended-Grenze) mit Frühwarnsignalen verankern.

Roter Faden

RPA ist kein Klick-Roboter, sondern ein *privilegierter Mitarbeiter*, der im Sekundentakt entscheidet und bucht. Wer ihn ohne Governance und ohne Audit Trail laufen lässt, gibt einer Software die Vollmachten eines Sachbearbeiters – ohne den menschlichen Bremshebel. Verantwortbare Automation heißt: Eignung prüfen, kontrolliert scheitern, transparent skalieren. (vgl. van der Aalst et al., 2018; Willcocks et al., 2017)

1. Einstieg: Warum RPA mehr ist als Klick-Magie

In vielen Organisationen begann RPA als Selbsthilfe-Werkzeug einzelner Fachbereiche: “Wir haben hier dreihundert Rechnungen pro Tag und drei Mitarbeitende, die nichts anderes tun als Kopieren aus E-Mail in ERP. Können wir das automatisieren?” Die Antwort eines RPA-Tools lautet typischerweise: “Ja, in zwei Wochen.” Was als pragmatische Antwort beginnt, kann zu einer schwer beherrschbaren Plattform mit hunderten Bots, instabilen Portalen und unklaren Verantwortlichkeiten heranwachsen. (Willcocks et al., 2017)

Die Frage ist daher nicht: *Können wir RPA?* Sondern: *Können wir RPA verantworten?*

1.1 Wofür RPA geeignet ist – und wofür nicht

RPA arbeitet typischerweise an der Benutzeroberfläche oder über strukturierte Schnittstellen. Es kopiert, klickt, liest – so wie ein Mensch es täte, nur schneller und fehlerärmer (in stabilen Fällen). Die typischen Eignungskriterien: (Scheppeler & Weber, 2020)

- **Regelbasiert.** Es gibt klare Wenn-dann-Regeln, keine Ermessensentscheidungen.
- **Stabile UI / stabile Schnittstellen.** Das Zielsystem ändert sich selten. Jede UI-Änderung kann den Bot brechen.
- **Klarer Input und Output.** Der Bot weiß, was hereinkommt, und was er produzieren soll.
- **Hohes Volumen.** Lohnt sich erst ab einer relevanten Stückzahl. Drei Vorgänge pro Monat sind kein RPA-Kandidat.
- **Wenige Ausnahmen.** Wenn jeder fünfte Fall anders ist, wird der Bot zur Sammelstelle für Sonderlocken.

Weniger geeignet sind Prozesse mit häufigen UI-Änderungen, vielen Sonderfällen, unklaren Regeln und hohem Interpretationsanteil. Hier hilft RPA allenfalls in Kombination mit AI-Komponenten (Document Understanding, Klassifikation) – aber das ist eine andere Disziplin, mit anderen Risiken.

1.2 RPA im BPM-Lifecycle

RPA steht nicht neben BPM, sondern *in* ihm. Im klassischen Lifecycle (Discovery – Analyse – Design – Automation – Monitoring – Optimization) ist RPA eine Automatisierungs-Option neben Workflow-Engines, ERP-Custom-Code oder API-Integration. Die Wahl hängt vom Reifegrad des Prozesses ab: (Dumas et al., 2018, Kap. 10)

- Unausgereifter, dokumentationsarmer Prozess → erst klären, dann automatisieren.
- Stabiler Prozess, stabile Systeme, API verfügbar → eher API-Integration.
- Stabiler Prozess, Legacy-System ohne API, hohes Volumen → klassischer RPA-Fall.

1.3 Die Versuchung des schnellen Erfolgs

RPA hat eine besondere Versuchung: *schnelle Erfolge*. Ein Pilot ist in zwei bis vier Wochen lauffähig, der ROI scheint klar, das Management ist beeindruckt. Das ist der Punkt, an dem Disziplin am wichtigsten ist – denn der zweite, dritte und zehnte Bot bringt die Skalierungsprobleme: (Smeets et al., 2019)

- **Schatten-IT.** Fachbereiche bauen ohne IT-Beteiligung. Niemand kennt den Bestand, niemand wartet, niemand hat Backups.
- **Fragile UI-Automation.** Ein UI-Update legt zehn Bots gleichzeitig lahm.

- **Credential-Risiko.** Bots brauchen Zugänge. Wenn die nicht zentral verwaltet sind, entstehen sicherheitsrelevante Lücken.
- **Fehlbuchungen im großen Maßstab.** Ein Bot, der einmal falsch programmiert ist, macht denselben Fehler tausendmal – in einer Geschwindigkeit, in der kein Mensch eingreifen kann.

Eselsbrücke: R-S-V-A

Vier Eignungskriterien für RPA, in Reihenfolge prüfen:

Regelbasiert – klare Wenn-dann-Logik?

Stabil – ändert sich UI/Prozess selten?

Volumen – relevante Stückzahl pro Monat?

Ausnahmen – niedrige Quote (idealerweise <10%)?

Wenn auch nur eine Antwort “nein” ist, lohnt sich die Eignung noch nicht.

2. Plattformüberblick: Studio, Orchestrierung, Run

RPA-Plattformen haben unterschiedliche Hersteller, aber sehr ähnliche Architekturen. Wer eine Plattform versteht, versteht sie alle – die Begriffe variieren, die Konzepte sind nahezu gleich.

(Lacity & Willcocks, 2018)

2.1 Studio / Designer: das Werkzeug zum Bauen

Im **Studio** (oder Designer) wird die Bot-Logik gebaut. Typische Bausteine: *Workflows* (eine Folge von Schritten), *Activities* oder *Actions* (einzelne Tätigkeiten wie “E-Mail lesen”, “Excel öffnen”, “Daten in Maske eintragen”), *Variablen*, *Bedingungen*, *Schleifen*. Wer schon einmal Workflow-Tools wie Camunda oder Power Automate genutzt hat, findet sich schnell zurecht.

Wichtige Disziplinen im Studio:

- **Naming Convention.** Activities, Workflows und Variablen folgen klaren Benennungsregeln. Sonst wird ein Bot mit 200 Schritten unwartbar.
- **Modularisierung.** Wiederkehrende Bausteine als Sub-Workflows. Wer alle Aktivitäten in einen Workflow packt, baut den klassischen “Spaghetti-Bot”.
- **Fehlerbehandlung.** Try/Catch um kritische Schritte, mit definierter Fehlerklasse.
- **Konfiguration extern.** Hostnames, Pfade, Schwellenwerte nicht im Code, sondern in einer Konfigurationsdatei oder im Orchestrator.

2.2 Orchestrierung: das Kontrollzentrum

Der **Orchestrator** (oder Control Room, Management Console) ist das Herz der Plattform. Er übernimmt: (Smeets et al., 2019)

- **Planung.** Welcher Bot läuft wann, auf welcher Maschine, mit welchem Zeitplan?
- **Queues.** Eingangs- und Ausgangswarteschlangen mit Items, Status, Priorität.
- **Credentials.** Zentral verwaltete Zugangsdaten. Bots greifen über Tokens darauf zu, nie über fest codierte Passwörter.
- **Monitoring.** Status der Bots, Fehlerraten, Durchsatz, Alerts.
- **Logs.** Was hat der Bot wann getan, mit welchem Input, mit welchem Output, mit welchem Fehler?

2.3 Run-Umgebung: wo der Bot lebt

Der eigentliche Ausführungsknoten – in attended Fällen am Mitarbeiterarbeitsplatz, in unattended Fällen auf einer Server- oder Cloud-VM. Drei Aspekte: (Scheppler & Weber, 2020)

- **Isolierung.** Jeder Bot läuft in einer isolierten Umgebung, damit ein abgestürzter Bot nicht andere mitreißt.
- **Ressourcen.** CPU, RAM, ggf. Bildschirmauflösung (wenn UI-Automation). Spielt mehr Bot-Volumen gegen Hardware-Kosten aus.
- **Verfügbarkeit.** Was passiert, wenn die Maschine ausfällt? Failover, Standby, oder akzeptiertes Risiko?

2.4 Low-Code vs. Enterprise

Auf dem Markt gibt es zwei Pole: Low-Code-/No-Code-Plattformen (oft auch Citizen Developer ansprechend) und Enterprise-Plattformen mit umfassender Governance. Beide haben ihre Berechtigung – aber sie skalieren unterschiedlich:

- **Low-Code.** Schnell, niedrige Einstiegshürde, gut für Pilotierung und kleinere Use Cases. Risiko: ohne Standards entsteht Wildwuchs.
- **Enterprise.** Höhere Einstiegshürde, dafür Governance, Audit, Skalierung mitgedacht. Risiko: bürokratisch, langsam, ohne Buy-in nicht akzeptiert.

3. Attended vs. Unattended: zwei Welten, eine Entscheidung

Die wichtigste strategische Entscheidung beim RPA-Einsatz ist die Wahl zwischen *attended* und *unattended* – oder einer abgestuften Kombination. Sie betrifft Betriebsmodell, Governance, Skalierbarkeit und Risiko.

3.1 Attended: Assistent am Arbeitsplatz

Ein **attended Bot** läuft am Arbeitsplatz eines Mitarbeitenden. Der Mitarbeitende triggert ihn (per Klick oder Hotkey), überwacht ihn, greift bei Problemen ein. Typische Anwendungen: Teilschritt-Assistenz (“Bitte fülle diese fünf Felder im ERP automatisch aus”), Datenkonsolidierung, repetitive Klick-Strecken.

- **Vorteile.** Geringeres Betriebsrisiko, niedrige Governance-Hürde, schnelle Lernkurve, Akzeptanz im Team (“der Bot hilft mir, ersetzt mich nicht”).
- **Nachteile.** Begrenzte Skalierung, abhängig von Anwesenheit, schwer auditierbar (es gibt nicht immer einen zentralen Log-Pfad).

3.2 Unattended: serverseitig, vollautomatisch

Ein **unattended Bot** läuft serverseitig (oder in der Cloud), ohne menschliche Anwesenheit. Er wird über Zeitpläne oder Queues angestoßen, arbeitet im Schichtbetrieb (auch 24/7), und meldet Ergebnisse über das Orchestrator-System zurück. (Lacity & Willcocks, 2018)

- **Vorteile.** Hohe Skalierung, kontinuierlicher Betrieb, klar auditierbar (alle Aktionen im zentralen Log), entkoppelt vom Arbeitsplatz.
- **Nachteile.** Höhere Anforderungen an Governance, Security, Exception Handling, Monitoring. Höhere initiale Kosten.